Evaluation of Multi-Layer Perceptron and Support Vector Machines for Text Classification on News Data

Deepak Joshi, MSc Data Science (FT)

INTRODUCTION

The accessibility of internet and the exponential growth of textual data created a need to categorise the text in the article, email, news etc into segment for documenting or further processing. With increasing volume and velocity of textual data it became impractical to manually label the text and the initial rule-based approaches were rigid and lacks adaptability. Hence, the machine learning (ML) approaches were needed to tackle the shortcomings of the earlier rigid approaches. Text classification is used in email spam filtering, sentiment analysis, search optimisation, recommending personalised news, analysing records, etc [1].

Text classification or Text Categorisation is the process of assigning the textual object to a set of labels. The process includes features extraction, where raw text is converted into a numeric format and then machine learning models are trained on them using pre-labelled datasets. These methods are fast, scalable and consistent way of text organisation without human intended bias [2].

Support Vector Machines (SVM)

SVM is a supervised ML algorithm used for classification of any set vectors, and it is quite popular used in text datasets because of its high robustness with high dimensional data. It classifies by determining the best decision boundary(hyperplane) to separate vectors. It achieves strong separation by maximizing the functional margins with support vectors. Using kernel trick, it can also classify the datapoints which are not easily separated by linear boundary by mapping data into higher dimensional space [3]. It is highly vulnerable to imbalanced data [2].

Multi-Layer Perceptron (MLP)

MLP is a neural network which has 3 types of layers: an input layer, one or more hidden layers and an output layer. MLP acts as a global approximator as with few hidden layers and continuous sigmoid activation function it can approximate almost any continuous function. The algorithm adjusts the internal weights and biases of the neurons to produce an output function closest true target function. The weights adjustments are done iteratively using Gradient Descent (back propagation) in three step process of propagation, error calculation and back propagation with weight changes. They are prone to overfitting complex data and might suffers from vanishing gradient problem [4].

Hypothesis

MLP being the global approximator will be able to learn semantic patterns better than SVM, but due larger parameters and complex modelling it might overfit.

DATASET

The Reuters-21578 dataset is a collection of the documents that appeared on the Reuters newswire in 1987 [5]**.** It consists of 21578 documents and their topics but after extracting data in usable form it has 11367 documents which we will use in our comparative study. This provides the opportunity to work on the muti-class classification problem. It consists of 120 unique topics associated with the documents and we will use top 15 topics for classification because dataset is so heavily imbalanced, the representation of documents across categories drops off sharply (Figure 1).

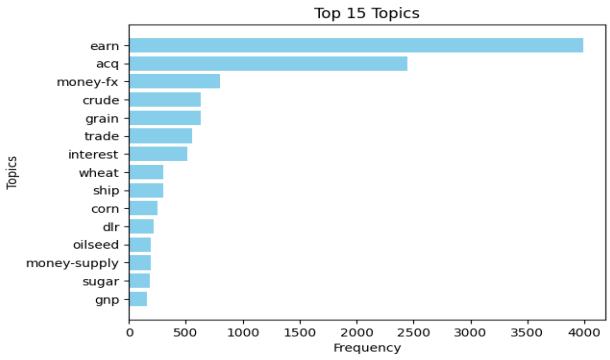


Fig 1. Number of documents with top 15 topic

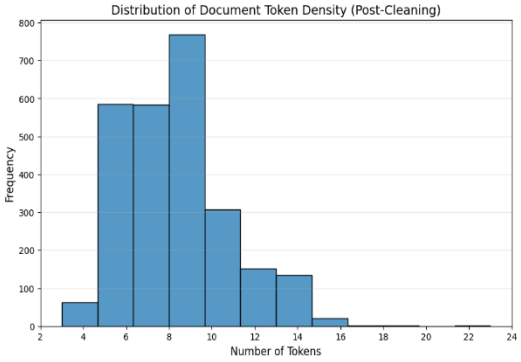


Figure 2. Document token distribution

The text of body and title of the article is merged to form a single column called text and then NLP preprocessing pipeline was implemented which includes links removal, Unicode removal, numeric removal and stemming of tokens. Fig 3. shows the word cloud of the text involved in some topics which shows the most frequent words in the text of that class.



Fig 3. Word cloud for randomly selected topics

METHEDOLOGY

For implementation and comparative analysis, the original dataset needs to be processed and converted from multi-label problem to single label problem (10 classes), to achieve this least frequent label from a list of article topics were extracted which improved data balance. First step was to extract the vectors

from the semantic words using a probabilistic term weighting technique to improve classification of minority class articles [6]. The max features were set to 1500 for TF-IDF. The data set was split into 70-15-15 split for training, validation and testing.

MLP Training Methodology

Firstly, the basic 2-hidden layer network (128x64, ReLU) was trained with default Adam optimiser using PyTorch with loss as CrossEntropy Loss, but it was overfitting (Fig. 4). To tackle that we tried dropout layer (0.5 & 0.3) and training took a toll but slight improvement in overfitting. After, that we implemented weight decay and momentum which resulted in train and validation loss difference reduction to 0.025 (Fig. 5). Model with more hidden layer resulted in more overfitting, hence decided to stick with 2-hidden layer and an extensive grid search (96 combinations) with selection metric as Jaccard accuracy which is more robust to unbalanced data.

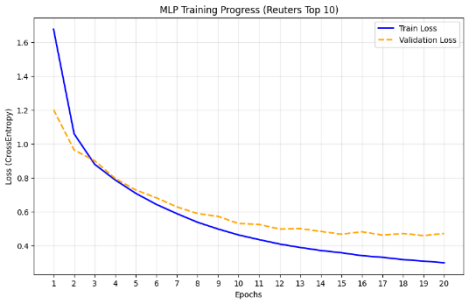


Fig. 4: MLP Default configuration loss

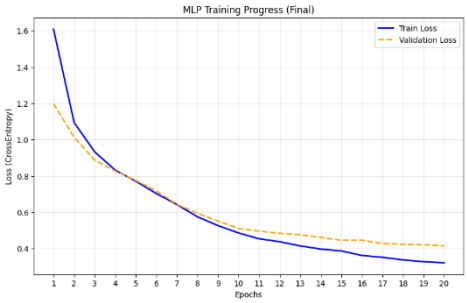


Fig. 5: MLP (dropout + decay) loss

SVM Training Methodology

Support vector machine provides different kernels to fit the data with degree of freedom and regularisation parameter [7]. First, we implemented the linear support vector classifier (LinearSVC) with "One-vs-Rest" classification and default settings (C=1, loss=hinge and max_iter = 10000) which provided the Jaccard accuracy of 63.30% but when tried SVC (kernel ='linear') which uses "One-vs-One" classification results dropped to 49%. LinearSVC built 10 classifiers whereas SVC (linear kernel) built 45 classifiers for 10 classes. We have implemented individual grid search for each kernel to find the optimal hyperparameters parameters.

Testing

The final testing was performed on the separate test set and four models were compared, 3 variations of SVM (LinearSVC, SVM {Linear} and SVM {RBF}) and MLP. Jaccard accuracy was used for evaluation of results. Classification report was published to for overall best performing model.

RESULTS AND EVALUATION

SVM and MLP uses different hyperparameter for model optimisation, the selection for the best model was done by considering both training and testing Jaccard accuracy. MLP with the default settings, performed well but happens to overfit with just 2 hidden layer as shown by the difference between Train Loss and Validation loss in Fig. 4. To improve the generalisation of the model, dropout layer was implemented after the input layer and the hidden layer, Fig. 5 shows the losses of the improved model which tackles overfitting.

hidden_dim	dropout	lr	beta1	weight_decay	mean_train_jaccard	mean_val_jaccard
(256, 128)	(0.3, 0.3)	0.001	0.95	0.0001	0.651185	0.616311
(256, 128)	(0.25, 0.2)	0.001	0.95	0.00001	0.673206	0.634095
(256, 128)	(0.3, 0.2)	0.001	0.95	0.0001	0.674969	0.634921
(256, 128)	(0.3, 0.3)	0.001	0.95	0.00001	0.677362	0.639091
(256, 128)	(0.25, 0.2)	0.001	0.95	0.0001	0.681199	0.642051
(256, 128)	(0.3, 0.3)	0.001	0.9	0.00001	0.692493	0.64586
(256, 128)	**(0.25, 0.2)**	**0.001**	**0.9**	**0.0001**	**0.672923**	**0.650664**
(256, 128)	(0.25, 0.2)	0.001	0.9	0.00001	0.71074	0.656237
(256, 128)	(0.3, 0.2)	0.001	0.95	0.00001	0.70469	0.65664
(256, 128)	(0.3, 0.2)	0.001	0.9	0.00001	0.707961	0.662302

Fig. 6: MLP Grid search (Top -10 results)

With optimised model, results on test set improved as it achieved f1-score(macro) of 0.82 and Jaccard score (macro) of 0.71. Fig 7 shows the f1-score and figure 8 shows the confusion matrix for the model for different class in this classification task and.

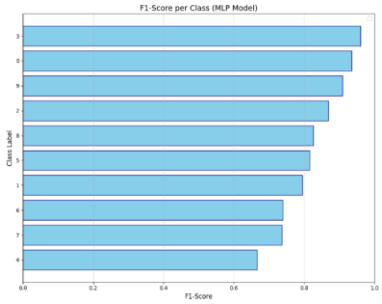


Fig. 7: F1 Score (MLP)

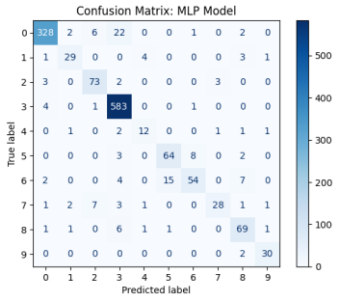


Fig. 8: Confusion Matrix (MLP)

To retrieve the optimal model hyperparameters, Grid search results (Fig. 8) were analysed and those setting were chosen which produces less difference in the train and test metric while keeping validation score sufficiently high.

Type	C	mean_train_score	mean_test_score
Linear Kernel	50	0.761998	0.671417
Linear Kernel	**10**	**0.677355**	**0.635704**
Linear Kernel	1	0.563026	0.543989
Linear Kernel	0.1	0.416801	0.415674
Linear Kernel	0	0.12968	0.130864
LinearSVC	100	0.841254	0.726839
LinearSVC	50	0.815148	0.725057
LinearSVC	10	0.758704	0.705126
LinearSVC	**5**	**0.731993**	**0.694751**
LinearSVC	1	0.678749	0.655771
LinearSVC	0.1	0.597977	0.590858
LinearSVC	0	0.48582	0.479668

Fig. 9: SVM (Kernel) Grid Search

C	gamma	mean_train_score	mean_test_score
5	scale	0.70089	0.647466
1	**scale**	**0.617528**	**0.581422**
0.1	scale	0.468481	0.464514
5	0.01	0.422533	0.421588
1	0.01	0.236765	0.23686
0.01	scale	0.200634	0.199211
5	0.001	0.098918	0.099122
1	0.001	0.086133	0.086104
1	auto	0.067366	0.067329
5	auto	0.065003	0.06469
0.1	0.01	0.057019	0.056993
0.01	auto	0.003556	0.003545
0.01	0.001	0.003556	0.003545
0.01	0.01	0.003556	0.003545
0.1	auto	0.003117	0.003117
0.1	0.001	0.003117	0.003117

Fig. 10: SVM (RBF) Grid Search

Surprisingly, SVM with LinearSVC implementation was faster to train and its results were comparable with other SVM kernels after tuning, so LinearSVC was chosen as the best performing SVM model. The final LinearSVC was used on test dataset and gave a Jaccard score (macro) of 0.69 an f1-score (macro) of 0.81.

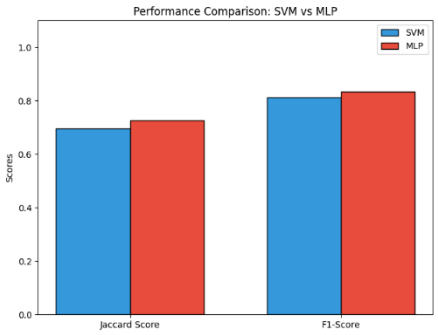


Fig. 11: comparison of SVM and MLP

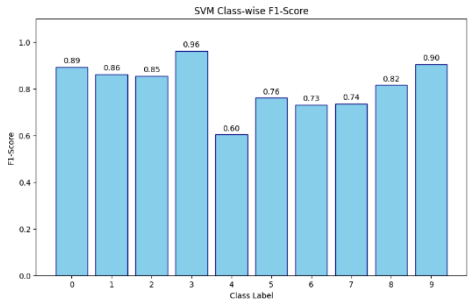


Fig. 12: F1 Score (SVM)

Both models performed competitively (Fig 11) and struggle heavily to classify the “class 4”, as SVM has f1-score of 0.60 where as MLP has 0.64 for “class 4”. The potential reason for the misclassification can be imbalanced dataset for that class, even after weighted TF-IDF implementation.

While optimising MLP, it took around 40 mins to run 96 combinations with 2 cores whereas SVM (RBF) took around 50 mins for running 36 combinations with 4 cores. This shows high computational cost associated with the SVM (RBF).

LESSONS LEARNED AND FUTURE WORK

In this study we have identified SVM offers good, computationally efficient performance for high-dimensional textual data, whereas MLP can learn complex nature of semantic data but sensitive to overfitting and needed optimal regularisation. Both models were impacted by the skewed class distribution, where minority classes were difficult to classify.

Weighted TF-IDF was found to be very beneficial as the standard TF-IDF was failed at correctly represent the minority classes and different text preprocessing can be done to improve the data quality for models to classify.

In SVM tuning model can be improved further with non-uniform class distribution and class weighting based on the dataset. From an MLP perspective, as we increased more layers overfitting increased, so regularisation can be improved with more computational power for running denser grid search.

REFRENCES

1. Hasan, S.A., Hussain, M.G., Protim, J., Rahman, M.M., Fahim, N., Chowdhury, M.Z. & Pritom, A.I. 2022, Classification of Multi-Labeled Text Articles with Reuters Dataset using SVM, IEEE.

2. Allam, H., Makubvure, L., Gyamfi, B., Graham, K.N. & Akinwolere, K. 2025, Text Classification: How Machine Learning Is Revolutionizing Text Categorization, MDPI AG.
3. Neural computing Lecture 4 slides (SVM)
4. Neural computing Lecture 2 slides (MLP)
5. “Reuters-21578 Text Categorization Collection - UCI Machine Learning Repository.” https://archive.ics.uci.edu/dataset/137/reuters+21578+text+categorization+collection
6. Liu, Y., Loh, H.T. & Sun, A. 2009, Imbalanced text classification: A term weighting approach, Elsevier BV.
7. “Sklearn.svm,” Scikit-learn. https://scikit-learn.org/stable/api/sklearn.svm.html